REA: Reverse Engineer Anything

Comprehensive Technical Overview & Architecture Guide • GitHub Repository Analysis

1. What is REA?

Reverse Engineer Anything (REA) is an open-source GitHub repository (`morluto/rea`) that viralized globally with over 115,000 stars. It functions as a specialized Model Context Protocol (MCP) server and Command Line Interface (CLI) bridge that seamlessly connects modern AI coding agents (such as Claude Code, Cursor, Codex, Pi, and Gemini CLI) directly with professional reverse-engineering tools like Ghidra, Hopper, IDA Pro, and browser automation engines.

> **Core Mission:** REA serves as an orchestration connector rather than an isolated sandbox. It allows coding agents to methodically inspect compiled binaries, Electron desktop apps, network packet captures (HAR), and web environments to extract module structures, pseudocode, and execution logs without manual tool overhead.

2. Why is REA Used?

Traditionally, reverse engineering required deep domain expertise in assembly, memory layouts, and arcane disassemblers. REA changes this by bridging AI intelligence with hard analysis engines, solving several major development hurdles:

Feature Reconstruction
Enables developers to study how specific features work in existing apps and replicate their mechanics cleanly in their own projects.

API Surface Discovery
Parses network captures (HAR / mitmproxy) to automatically build precise API route inventories, parameter maps, and mock specs.

3. How REA Works

REA operates by establishing a standardized bridge between AI coding agents and underlying binary/network analysis engines using JSON-RPC via the Model Context Protocol (MCP). When an agent needs to inspect a target, it invokes REA tools to query the environment.

Workflow Stage	Agent Responsibility	REA Engine Action
1. Isolation	Isolates a single granular feature or function.	Configures target scope and prevents broad scans.
2. Inspection	Calls MCP tools (e.g., `inspect_binary`).	Queries Ghidra/Rizin/Hopper for symbol & control-flow data.
3. Tagging	Evaluates findings with explicit confidence levels.	Labels findings strictly as `seen`, `inferred`, or `unknown`.
4. Rebuilding	Writes clean, independent implementation code.	Executes automated test harness verification.

The Seen / Inferred / Unknown Framework

To prevent AI hallucination during complex binary analysis, REA enforces a strict evidence discipline. Every conclusion drawn by the agent must be categorized into one of three distinct confidence buckets:

> **Evidence Rigor:** `Seen` data represents direct compiled bytes or captured traffic. `Inferred` data represents logic deduced from surrounding control flow. `Unknowns` are explicitly documented gaps that require targeted fuzzing or further tracing.

4. Detailed Setup & Installation Guide

Installing REA takes only a few minutes when using Node.js and npm. Follow these exact steps to install the toolkit and register its MCP server with your preferred coding agent.

Step 1: Verify Node.js Prerequisites

REA requires a modern Node.js runtime (Node 22.19+ or 24.11+). Open your terminal and check your environment:

```
node --version
npm --version
```

Step 2: Install REA Globally via npm

Install the official package from the npm registry to make the global `rea` command and MCP server available across your development environment:

```
npm install --global rea-agents
```

Step 3: Configure Your AI Coding Agent

REA features an automated setup utility that injects necessary MCP server bindings into your agent configuration (supporting Claude Code, Cursor, Pi, Codex, and Gemini CLI):

```
rea setup --client claude-code
```

Review the proposed changes when prompted, approve the configuration, and restart your coding agent to load the active tools.

5. System Verification & Doctor Command

After configuring your agent, verify that all MCP communication channels and binary provider paths are operating correctly by executing the built-in diagnostic tool:

```
rea doctor --client claude-code
```

This command inspects local configuration files, verifies JSON-RPC streaming, and reports any missing dependencies or permission errors.

6. Keeping REA Updated

Because REA is actively maintained and receives frequent updates, keeping your CLI and agent rules synchronized is essential for stability:

```
rea update
```

> **Security Best Practice:** REA is a local connector, not an isolated sandbox. Running untrusted binaries will execute parsing routines under your active user permissions. Always analyze unknown executables inside an isolated virtual machine or container environment.